
|| DIODEGUARD BLUEPRINT v1.0 ||

|| AI-Driven Passive Cyber Threat Detection Pipeline ||

|| SIH Problem Statement: SIH26145 ||

1. PROJECT OVERVIEW

Project Name: DiodeGuard

Version: 1.0.0

Domain: Cybersecurity / Critical National Infrastructure (CNI)

Problem Statement: SIH26145 — AI-based passive detection of cyber threats in unidirectional IP traffic

One-Line Summary:

A real-time AI/ML pipeline that ingests one-directional IP traffic from a hardware data diode and detects, classifies, and scores cybersecurity threats using only passively collected metadata — with zero payload decryption and zero return path.

2. PROBLEM CONTEXT

2.1 What is a Data Diode?

A **hardware data diode** is a physically enforced one-way network bridge. It mirrors all traffic flowing across a gateway/peering link into a separate monitoring enclave. The critical property is:

- Traffic flows **IN** to the enclave →  Allowed
- Traffic flows **OUT** from the enclave →  Physically impossible

This means:

- The monitoring system can **observe** everything crossing the link.

- The monitoring system **cannot** send probes, complete handshakes, push mitigation commands, or communicate back in any way.

2.2 Why Does This Matter?

Critical infrastructure operators (Power Grids, Telecom, Defense Networks, SCADA/ICS) use data diodes to ensure that even if the monitoring/analytics system is fully compromised by an attacker, they have **zero lateral movement** path back into the production network.

2.3 The Challenge

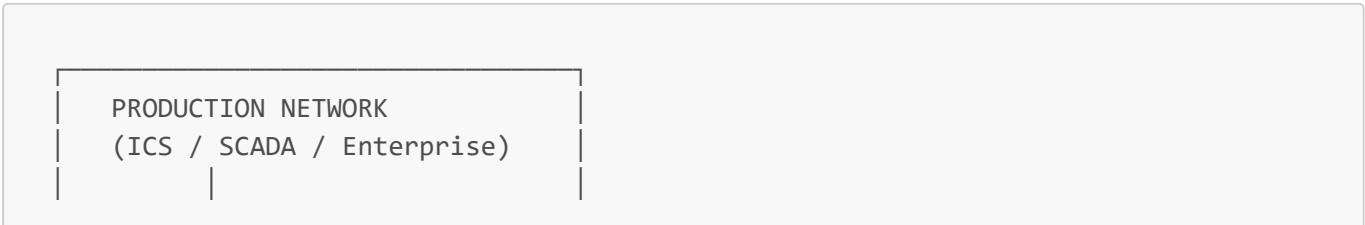
Build an AI pipeline that works **entirely within these constraints** — detecting threats from metadata alone, in real-time, without ever touching payloads or sending a single packet back.

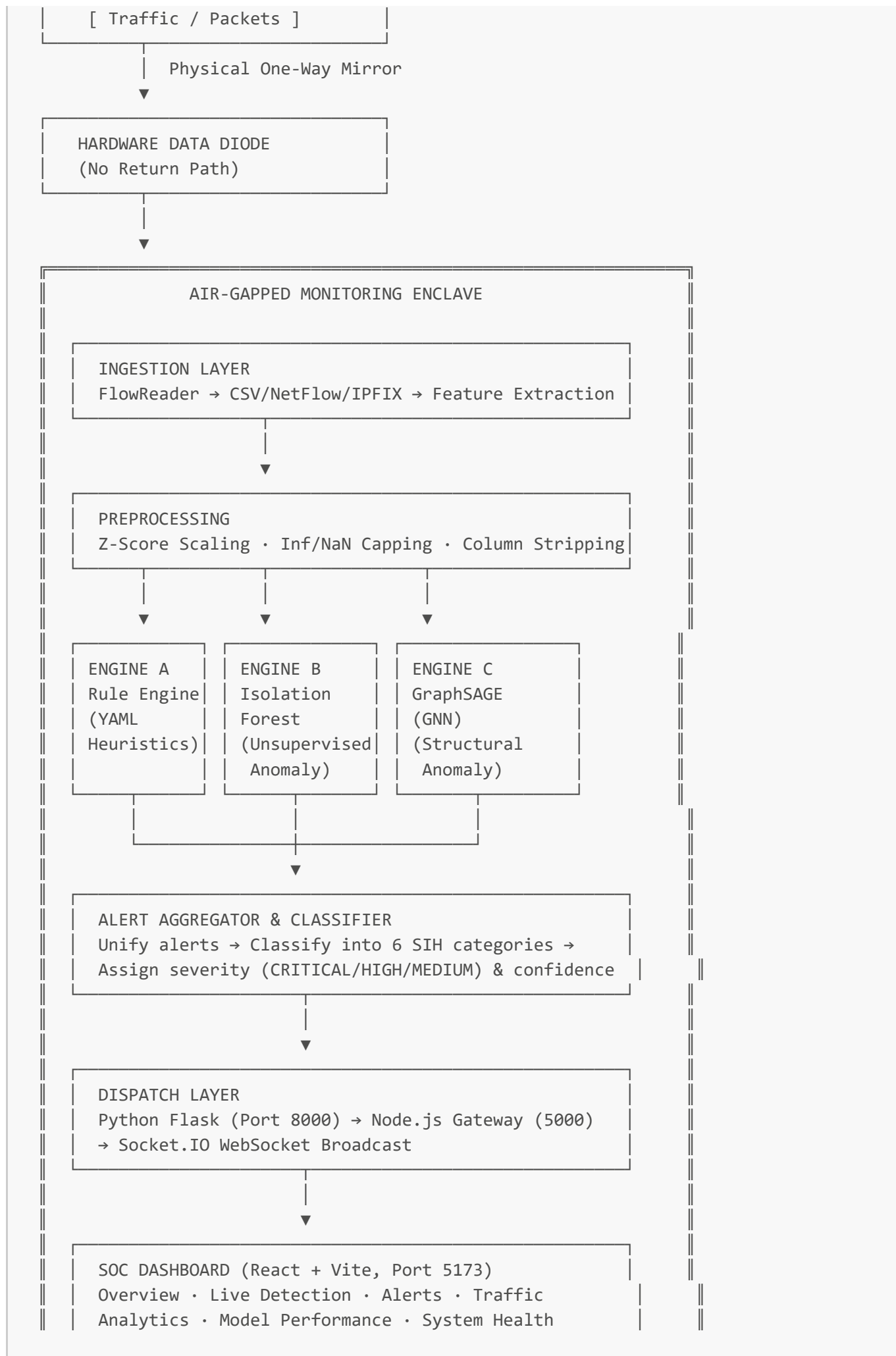
3. THREAT CATEGORIES (6 SIH-MANDATED)

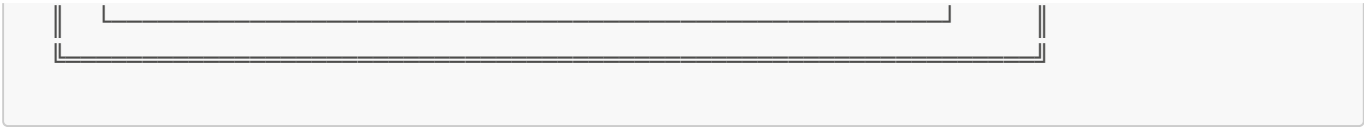
#	Threat Type	Detection Method Used	Engine
1	Volumetric / Protocol DDoS (SYN floods, UDP reflection, spoofed-source floods)	Flow-level rate thresholds, Flow Packets/s , Flow Bytes/s , source-IP entropy	Rule Engine
2	Botnet C2 Beaconsing (periodic callbacks to C2 servers)	Flow IAT Mean/Std periodicity analysis, repetitive flow duration patterns	Rule Engine + GraphSAGE
3	DGA Domains & DNS Tunnelling (entropy/n-gram anomalies in DNS queries)	DNS port (53) flow behavioral analysis, query length anomalies via flow metadata	Rule Engine
4	Malware inside Encrypted Sessions (TLS/QUIC metadata analysis)	Packet-size sequences, Fwd/Bwd Packet Length distributions, timing patterns — NO decryption	Rule Engine + Isolation Forest
5	Reconnaissance & Port Scanning (fan-out from single source)	Rapid small-packet flows to diverse destination ports, fan-out ratio detection	Rule Engine
6	Data Exfiltration (asymmetric outbound volume)	Down/Up Ratio anomalies, unusual outbound-to-inbound byte ratios	Rule Engine + Isolation Forest

4. SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram







4.2 Technology Stack

Layer	Technology	Purpose
ML Framework	PyTorch + PyTorch Geometric	GraphSAGE Graph Neural Network
ML Library	Scikit-Learn	Isolation Forest unsupervised anomaly detection
Rule Engine	Custom Python + YAML	Heuristic signature-based threat matching
Backend API	Python Flask	ML inference orchestration, REST endpoints
Gateway	Node.js + Express	WebSocket relay, Socket.IO broadcast
Frontend	React 19 + Vite 8	SOC Dashboard SPA
Charts	Recharts	Real-time data visualizations
Animations	Framer Motion	Smooth UI transitions
Styling	Tailwind CSS	Dark cyber-neon theme
System Monitoring	psutil	CPU/Memory/Throughput tracking

5. MODULE BREAKDOWN

5.1 Ingestion Module (backend/app/ingestion/)

- **FlowReader** (flow_reader.py): Reads the CSV dataset row-by-row as a Python generator, simulating a real-time stream. Each row is mapped to a **FlowRecord** dataclass containing: src_ip, dst_ip, src_port, dst_port, protocol, duration, forward/backward packet counts, byte counts, IAT statistics, and a raw_features dictionary containing all 77 numeric feature columns.
- **FlowRecord Schema** (schemas/flow.py): Python dataclass defining the canonical flow representation used across all engines.

5.2 Rule Engine (backend/app/detection/rules/)

- **RuleEngine** (rule_engine.py): Loads YAML rule files from the rules/ directory at startup. Each rule defines threshold conditions on flow features. On each flow, it evaluates all rules and returns a list of triggered rule names.
- **YAML Rules** (7 active rules):
 - port_scan.yaml — Detects rapid fan-out to diverse ports
 - ddos_volume.yaml — Detects volumetric flood patterns
 - c2_beacon.yaml — Detects periodic beaconing behavior
 - dns_tunnel.yaml — Detects DNS tunnelling via port 53 anomalies
 - data_exfil.yaml — Detects asymmetric outbound byte ratios
 - encrypted_malware.yaml — Detects malware via packet-size distributions
 - generic_anomaly.yaml — Catch-all for suspicious flow patterns

5.3 Isolation Forest ([backend/app/detection/isolation_forest/](#))

- **AnomalyDetector** ([isolation_forest.py](#)): Wraps Scikit-Learn's `IsolationForest`.
 - **Training**: Dynamically trained on the first 2,000 flows (assumed benign baseline) with `contamination=0.05` (5% outlier threshold).
 - **Inference**: Takes a single flow's feature vector, predicts whether it is an inlier (benign) or outlier (anomalous).

5.4 GraphSAGE GNN ([backend/app/detection/graphsage/](#))

- **GraphSAGEDetector** ([model.py](#)): A 2-layer `SAGEConv` neural network (PyTorch Geometric).
 - Layer 1: `SAGEConv(input_dim → 128)` + ReLU + Dropout
 - Layer 2: `SAGEConv(128 → 2)` (binary classification: benign vs malicious)
- **Graph Construction** ([graph_construction.py](#)): Builds a K-Nearest Neighbors (K=3) graph dynamically. The new flow node is attached to a rolling window of the last 50 background flows. Edge weights are determined by Euclidean distance in feature space.
- **Inference**: Forward pass through the GNN produces a 2-class softmax probability. If `P(malicious) > 0.60`, a `STRUCTURAL_ANOMALY` alert is raised.

5.5 Orchestrator ([backend/app/orchestrator.py](#))

The central brain of the system. For each incoming flow:

1. Passes the flow through all 3 engines concurrently.
2. Records per-engine latency (milliseconds).
3. Classifies triggered alerts into the 6 SIH threat categories.
4. Tracks comprehensive statistics: per-IP flows, protocol distribution, port distribution, total bytes, and the full alert history.
5. POSTs structured alert payloads to the Node.js Gateway.

5.6 Flask API ([backend/app/main.py](#))

Endpoint	Method	Description
/health	GET	System health: uptime, CPU, memory, engine status, hostname
/metrics	GET	Overview metrics: total flows, total alerts, threat distribution
/replay	POST	Start replay: <code>{count: N, delay: D}</code> to stream N flows with D-second intervals
/replay/stop	POST	Stop the active replay thread
/replay/status	GET	Check if a replay is currently running
/alerts	GET	Full alert history with optional <code>?severity=</code> and <code>?category=</code> filters
/traffic	GET	Traffic analytics: protocols, top talkers, top ports, byte counts
/models	GET	Per-engine performance: detections, calls, avg latency, detection rate

5.7 Node.js Gateway ([node_backend/server.js](#))

- Express server on port 5000.
- Exposes `POST /api/log-activity` which the Python backend calls to push alerts.
- Broadcasts each alert to all connected React clients via `Socket.IO` event `new_activity`.

5.8 React SOC Dashboard (`frontend/src/`)

Page	Route	Data Source	Description
Overview	<code>/dashboard</code>	<code>GET /metrics</code> (1s poll)	Top-level KPIs: flows, alerts, threat score, engine count. Area chart of threat distribution over time. Live activity stream via Socket.IO.
Live Detection	<code>/live</code>	Socket.IO + <code>GET /metrics</code> (2s poll)	Start/Stop Replay controls. Real-time alert table with AnimatePresence animations. Severity-coded badges.
Alerts	<code>/alerts</code>	<code>GET /alerts</code> (2s poll)	Searchable, filterable alert table. Expandable rows showing evidence. Severity and category dropdown filters.
Traffic Analytics	<code>/traffic</code>	<code>GET /traffic</code> (3s poll)	Protocol PieChart, Top Talkers table, Top Ports BarChart. Per-IP threat scoring.
Model Performance	<code>/models</code>	<code>GET /models</code> (3s poll)	Per-engine cards: detections, calls, avg latency, detection rate. Threat category distribution breakdown.
System Health	<code>/health</code>	<code>GET /health</code> (2s poll)	Service status cards (Python/Node/React). CPU and Memory gauges. Engine status list. Device info.

6. STANDARDIZED ALERT SCHEMA (SIH REQUIREMENT)

Every alert produced by any engine is normalized into this JSON structure:

```
{
  "id": "ALERT-1714567890123",
  "timestamp": "2026-08-29T14:30:00.000Z",
  "flow_id": "a3b2c1d4-e5f6-7890-abcd-ef1234567890",
  "threat_class": "STRUCTURAL_ANOMALY",
  "category": "Anomaly",
  "severity": "CRITICAL",
  "source_ip": "192.168.10.15",
  "dest_ip": "203.0.113.5",
  "src_port": 49152,
  "dst_port": 443,
  "confidence": 95.8,
  "engine": "GraphSAGE",
  "evidence": "Anomalous graph structure detected in traffic network",
  "status": "NEW"
}
```

Field Definitions:

- **id** — Unique alert identifier (epoch milliseconds)
- **timestamp** — ISO 8601 time of detection
- **flow_id** — UUID of the network flow that triggered the alert
- **threat_class** — Specific threat signature name
- **category** — One of: DDoS, Port Scan, C2 Beacon, DNS Tunnel, Anomaly, Data Exfil, Encrypted Malware
- **severity** — CRITICAL / HIGH / MEDIUM
- **confidence** — Percentage confidence score (0-100)
- **engine** — Which detection engine triggered: Rule Engine, Isolation Forest, or GraphSAGE
- **evidence** — Human-readable explanation for SOC analyst triage
- **status** — NEW / INVESTIGATING / RESOLVED

7. DATASET & FEATURE ENGINEERING

7.1 Dataset

- **Name:** CIC-IDS2017 (Canadian Institute for Cybersecurity)
- **File:** `datasets/combinenew.csv`
- **Content:** Pre-extracted network flow features containing both benign and attack traffic (DDoS, Port Scan, Brute Force, Web Attack, Infiltration, Botnet).
- **Note:** In production, raw PCAP data mirrored by the data diode would be converted to this format by Zeek or CICFlowMeter.

7.2 Feature Categories (77 numeric features)

Category	Example Features	Purpose
Volumetric	Flow Bytes/s, Flow Packets/s, Total Fwd Packets	DDoS / flood detection
Packet Size	Fwd Packet Length Max/Min/Mean/Std, Bwd Packet Length Mean	Encrypted malware fingerprinting
Temporal/IAT	Flow IAT Mean/Std, Fwd IAT Mean, Bwd IAT Mean	C2 beaconing periodicity
Asymmetry	Down/Up Ratio, Subflow Fwd/Bwd Bytes	Data exfiltration detection
TCP Flags	SYN Flag Count, FIN Flag Count, PSH Flag Count, ACK Flag Count	Protocol abuse detection
Bulk Transfer	Fwd Avg Bytes/Bulk, Bwd Avg Bulk Rate	Tunnelling anomalies
Window	Init_Win_bytes_forward, Init_Win_bytes_backward	OS fingerprinting hints

8. PROJECT STRUCTURE

```

DiodeGuard/
├── README.md           # Setup guide & teammate onboarding
├── ARCHITECTURE.md     # Technical architecture & ML methodology

```

```

├── PRESENTATION_CONTENT.md           # SIH PPT content guide
├── DiodeGuard Blueprint.md           # This file – full system blueprint
├── run.bat                           # One-click launcher (Windows)
├── .gitignore                        # Git exclusions
├── backend/                          # Python ML Backend
│   ├── app/
│   │   ├── main.py                  # Flask API server (port 8000)
│   │   ├── orchestrator.py          # 3-Engine pipeline orchestrator
│   │   ├── ingestion/
│   │   │   └── flow_reader.py        # Streaming CSV flow reader
│   │   ├── schemas/
│   │   │   └── flow.py               # FlowRecord dataclass
│   │   └── detection/
│   │       ├── rules/
│   │       │   ├── rule_engine.py    # YAML rule evaluator
│   │       │   └── rules/             # 7 YAML signature files
│   │       ├── isolation_forest/
│   │       │   └── isolation_forest.py # Scikit-Learn anomaly detector
│   │       └── graphsage/
│   │           ├── model.py           # 2-layer SAGEConv GNN
│   │           ├── graph_construction.py # K-NN graph builder
│   │           └── config.py          # Model hyperparameters
├── node_backend/                     # Node.js WebSocket Gateway
│   ├── server.js                    # Express + Socket.IO (port 5000)
│   └── package.json
├── frontend/                         # React SOC Dashboard
│   ├── src/
│   │   ├── App.jsx                  # Router configuration
│   │   ├── components/
│   │   │   ├── Layout.jsx            # Sidebar + Content wrapper
│   │   │   └── Sidebar.jsx           # Navigation sidebar
│   │   └── pages/
│   │       ├── Dashboard.jsx         # Overview page
│   │       ├── LiveDetection.jsx      # Real-time detection feed
│   │       ├── AlertsPage.jsx         # Alert management table
│   │       ├── TrafficAnalytics.jsx   # Traffic analysis charts
│   │       ├── ModelPerformance.jsx   # Engine performance metrics
│   │       └── SystemHealth.jsx       # System monitoring
│   ├── package.json
│   └── index.html
├── datasets/
│   └── combinenew.csv                # CIC-IDS2017 flow dataset (git-ignored)

```

9. HOW TO RUN (QUICK START)

Prerequisites

- **Python 3.8+** with: Flask, flask-cors, pandas, numpy, scikit-learn, torch, torch-geometric, requests, psutil
- **Node.js 18+** with: express, socket.io, cors
- **npm** (comes with Node.js)

Step-by-Step

```
# 1. Clone the repository
git clone https://github.com/YourUsername/DiodeGuard.git
cd DiodeGuard

# 2. Place the dataset
# Download CIC-IDS2017 and place as datasets/combine_new.csv

# 3. Install frontend dependencies
cd frontend && npm install && cd ..

# 4. Install Node gateway dependencies
cd node_backend && npm install && cd ..

# 5. Install Python dependencies
cd backend && pip install flask flask-cors pandas numpy scikit-learn torch
requests psutil && cd ..

# 6. Run everything (Windows)
run.bat

# OR run manually in 3 terminals:
# Terminal 1: cd backend && python app\main.py
# Terminal 2: cd node_backend && node server.js
# Terminal 3: cd frontend && npm run dev
```

Accessing the Dashboard

Open <http://localhost:5173> → Click **Live Detection** → Click **Start Replay**

10. SYSTEM WALKTHROUGH & USAGE GUIDE

This section explains exactly how the system behaves when demonstrating the prototype to stakeholders or judges.

Step 1: The Initial State (Idle)

When you first open the dashboard at <http://localhost:5173>, the system is in an idle, "listening" state.

- The **Overview** page shows 0 flows and 0 alerts.
- The **System Health** page shows the ML Engine, Gateway, and React frontend are all **ONLINE**, but throughput is at **0.0 FPS**.
- This represents a monitoring enclave that is successfully connected to the data diode but waiting for traffic.

Step 2: Triggering the Ingestion (Start Replay)

To simulate live traffic crossing the data diode, navigate to the **Live Detection** tab and click **Start Replay**.

What happens under the hood:

1. The React frontend sends a **POST /replay** request to the Python Flask API.
2. The Python backend spawns a background Daemon Thread.
3. The **FlowReader** begins reading **combinenew.csv** line-by-line, parsing network metadata into **FlowRecord** objects.
4. Each flow is sequentially pushed into the **DiodeGuardOrchestrator** for processing.

Step 3: The Threat Detection Pipeline

As flows are processed (at roughly 1,500+ flows per second), they hit the 3-Engine ML Core simultaneously:

1. **Rule Engine:** Checks if the flow matches any hardcoded YAML thresholds (e.g., *Is this a port scan?*).
2. **Isolation Forest:** Checks if the flow's numerical vector is a statistical outlier compared to the benign baseline.
3. **GraphSAGE:** Attaches the flow to the temporal K-NN graph and checks if its structural connections resemble a botnet or C2 beacon.

If an anomaly is detected: The orchestrator packages it into the Standardized Alert JSON Schema and POSTs it to the Node.js Gateway, which immediately broadcasts it to the frontend via Socket.IO.

Step 4: Visualizing the Live Data

While the replay is running, every page on the dashboard comes alive with real data:

- **Live Detection:** Threat alerts slide into the table in real-time. You will see severity badges (CRITICAL, HIGH, MEDIUM) dynamically populating.
- **Overview:** The "Flows Analyzed" counter skyrockets. The Threat Distribution Area Chart begins graphing the exact types of attacks happening second-by-second. The Activity Stream at the bottom scrolls with text logs of the detections.
- **Traffic Analytics:** The Protocol Pie Chart shifts as TCP/UDP ratios change. The Top Talkers table populates with the specific IP addresses sending the most traffic and generating the most alerts.
- **Model Performance:** You can see exactly how many flows each ML model has processed, their detection rates, and their live inference latency (e.g., GraphSAGE running at ~3.5ms per flow).
- **Alerts Page:** SOC Analysts can now use the dropdowns to filter the historical alerts by Severity or Category, and click on individual rows to expand the forensic evidence.

Step 5: Stopping the Replay

Clicking **Stop Replay** (or waiting for the batch to finish) halts the **FlowReader**. The system retains all calculated metrics and alert history for post-incident analysis, simulating the end of a traffic capture.

11. ARCHITECTURAL CONSTRAINTS COMPLIANCE

SIH Constraint	Requirement	DiodeGuard Implementation	Status
Read-Only Ingest	No return path, no live queries, no inline blocking	FlowReader is a one-way generator. Zero outbound network calls to traffic source.	✓
No Payload Decryption	TLS/QUIC analyzed from metadata only	Uses packet-size distributions, timing, and byte ratios. Zero DPI.	✓
Streaming, Not Batch	Bounded latency, incremental processing	Per-flow inference with <5ms average latency per engine.	✓
Defined Throughput	State and demonstrate traffic rate	Dashboard displays live <code>throughput_fps</code> . Target: 1,500+ flows/sec.	✓
Standardized Alert Schema	Structured records with timestamp, flow ID, threat class, confidence, evidence	Full JSON schema with all required fields.	✓

12. TEAM CONTRIBUTION AREAS

Area	Skills Required	Key Files
ML / Data Science	Python, PyTorch, Scikit-Learn	<code>orchestrator.py</code> , <code>model.py</code> , <code>isolation_forest.py</code>
Rule Engineering	YAML, Cybersecurity domain knowledge	<code>rule_engine.py</code> , <code>rules/*.yaml</code>
Backend API	Python Flask, REST APIs, Threading	<code>main.py</code> , <code>flow_reader.py</code>
Frontend UI	React, Tailwind CSS, Recharts	<code>pages/*.jsx</code> , <code>Sidebar.jsx</code>
DevOps / Integration	Node.js, Socket.IO, Git	<code>server.js</code> , <code>run.bat</code> , <code>.gitignore</code>
Documentation / PPT	Technical writing, Presentation	<code>ARCHITECTURE.md</code> , <code>PRESENTATION_CONTENT.md</code>

DiodeGuard v1.0 — Built for Smart India Hackathon 2026